

PWM Generation for LED Brightness Control Using STM32

This project demonstrates how to generate a PWM signal using the STM32 Timer peripheral to control the brightness of an LED.

The PWM signal is generated using the STM32 HAL library and configured through STM32CubeMX/STM32CubeIDE.

1. STM32CubeMX / IOC Configuration

Select Timer

Open the `.ioc` file in STM32CubeMX or STM32CubeIDE.

Select the required Timer peripheral.

Example:

```
```text id="h5j8r0" TIM2
```

```
Configure Clock Source
```

Select the appropriate clock source depending on the STM32 board and clock configuration.

```
```text id="0uh1c3"
```

Clock Source → Internal Clock / External Clock

Configure PWM Channel

Select the required Timer channel and configure it for PWM generation.

Example:

```
```text id="5om7xy" Channel → PWM Generation CH1
```

For example:

```
```text id="q3w2mj"
TIM2 → Channel 1 → PWM Generation CH1
```

The actual Timer and PWM output pin depend on the STM32 microcontroller and hardware configuration.

2. Timer Parameter Settings

The PWM frequency determines how fast the LED is switched ON and OFF.

The PWM period is calculated using:

```
```text id="a8f8y4" T = 1 / f
```

For example, if the desired PWM frequency is **1 kHz**:

```
```text id="u8m2gu"
T = 1 / 1000
T = 0.001 seconds
T = 1 ms
```

If the timer is configured with a **1 µs timer tick**, the Counter Period can be:

```
```text id="1g5l5f" Counter Period = 1000 - 1 = 999
```

Therefore:

```
```text id="u2zv4d"
Prescaler      → Timer Clock / 1 MHz - 1
Counter Mode   → Up
Counter Period → 999
Auto Reload    → Enable
PWM Mode       → PWM Mode 1
Pulse          → Initial Compare Value
```

3. PWM Duty Cycle and LED Brightness

The PWM duty cycle controls the average power supplied to the LED and therefore controls its apparent brightness.

With:

```text id="7o2n6j" ARR = 999

the Compare value can be changed to control brightness.

### 100% Brightness

```
```c id="u6k4p0"  
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 999);
```

Approximate duty cycle:

```text id="5j5q7f" 100%

### 75% Brightness

```
```c id="x8j3h2"  
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 750);
```

Approximate duty cycle:

```text id="p8q6j2" 75%

### 50% Brightness

```
```c id="k1j5m0"  
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
```

Approximate duty cycle:

```text id="3l9h7z" 50%

### 25% Brightness

```
```c id="b6v2c8"  
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 250);
```

Approximate duty cycle:

```text id="j9x4n3" 25%

```
0% Brightness
```

```
```c id="q7m2k5"  
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
```

Approximate duty cycle:

```text id="t6n4p2" 0%

The relationship is approximately:

```
```text id="5qv3x1"  
Compare Value  
|  
▼  
PWM Duty Cycle  
|  
▼  
Average LED Power  
|  
▼  
LED Brightness
```

4. PWM Generation in `main.c`

Start PWM generation using:

```
```c id="j7v2k4" HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

This starts PWM output on:

```
```text id="0j7p9s"  
TIM2 Channel 1
```

The Timer and Channel depend on the configuration selected in the `.ioc` file.

Change LED Brightness

The PWM duty cycle can be changed using:

```
```c id="3m5v8k" __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, VALUE);
```

For example:

```
```c id="h4x8n2"
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 999);
```

sets approximately:

```
```text id="x8c3m1" 100% brightness
```

Similarly:

```
```c id="f5q2z7"
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
```

sets approximately:

```
```text id="j8m4k6" 50% brightness
```

```

5. Complete PWM LED Example

The following example continuously changes the LED brightness:

```c id="a6n3w8"
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

while (1)
{
    /* 100% brightness */

    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 999);

    HAL_Delay(2000);

    /* 75% brightness */
```

```

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 750);

HAL_Delay(2000);

/* 50% brightness */

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);

HAL_Delay(2000);

/* 25% brightness */

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 250);

HAL_Delay(2000);

/* 0% brightness */

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);

HAL_Delay(2000);
}

```

The LED brightness changes in the following sequence:

100% ↓ 75% ↓ 50% ↓ 25% ↓ 0% ↓ Repeat

```

---

# 6. Check PWM Configuration

If the LED does not glow or the PWM output does not work, check the following
file:

stm32f4xx_hal_msp.c

```

Verify that the required Timer peripheral and GPIO pin are initialized correctly.

Check:

- Timer clock is enabled.
- GPIO clock is enabled.
- PWM GPIO pin is initialized.
- GPIO is configured as **Alternate Function**.
- Correct Alternate Function is selected.
- Correct Timer Channel is selected.
- Physical LED pin matches the configured Timer Channel.
- `HAL_TIM_PWM_Start()` is called.
- Timer Prescaler is correct.
- Counter Period is correct.
- PWM Mode is set to PWM Mode 1.

The PWM GPIO should typically be configured as:

GPIO Mode → Alternate Function Alternate Function → Correct TIMx_CHy

7. PWM LED Summary

Example Timer configuration:

```text id="c5v9x2"

PWM Frequency = 1 kHz

Timer Tick = 1  $\mu$ s

Counter Period = 999

Duty cycle control:

Compare Value = 0 → 0% → LED OFF Compare Value = 250 → 25% → Low brightness

Compare Value = 500 → 50% → Medium brightness Compare Value = 750 → 75% → High brightness

Compare Value = 999 → ~100% → Maximum brightness

The PWM output is controlled using:

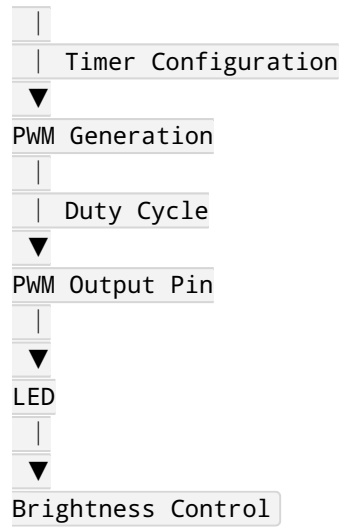
```c id="p4w8m2"

`__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, VALUE);`

Overall operation:

text id="q7x2m9"

STM32



Note: The perceived brightness of an LED is not always perfectly linear with PWM duty cycle because human vision has a nonlinear response. For advanced LED dimming, gamma correction can be used.